# Metro Property Shop Voice Agent

Complete Project Documentation

*A full narrative of what was built, how, every real bug found and fixed, deployment, and prepared answers to likely interview questions.*

# 1. What This Project Is

An end-to-end bilingual (Hindi + English) AI voice agent for a real estate business. It answers inbound calls, places outbound advertisement calls, understands what a caller wants, matches it against real listings, schedules site visits, and captures leads - all over an actual phone number, running on a free/low-cost stack. It is live and deployed, not a local demo.

GitHub: github.com/HamzaAITech/metro-property-shop-voice-agent

Live phone number: +1 (659) 234-7944

Live dashboard: metro-property-shop-voice-agent-production.up.railway.app

# 2. Why It Was Built

**Portfolio differentiation.**

Most AI portfolio projects are English-only text chatbots. Real-time, latency-sensitive, multilingual voice is a genuinely rarer and harder skill to demonstrate.

**Built for a real constraint, not a fictional one.**

Built unsolicited for a real local property shop - no client spec existed, so the call flow, scope, and features were designed from research into what a property shop's front desk actually needs, not copied from a brief.

**Deliberate scope sequencing.**

Started with a bounded, single-business version before considering a broader multilingual 'rural services' agent (banking, agriculture, government schemes) as a future direction - proves incremental scoping over over-building.

# 3. What The Agent Actually Does

- Greets the caller (or, on an outbound call, proactively introduces itself - the system prompt is direction-aware, so it never says the backwards 'thanks for calling' on a call the business placed).
- Handles casual small talk naturally (e.g. 'Kaise ho?') before steering back to business, instead of giving a robotic, out-of-context reply.
- Determines intent: buy vs. rent, property type, budget, locality.
- Matches against a real listings dataset and describes 1-2 relevant options - with an explicit arithmetic check against the stated budget before ever claiming a listing 'fits' (see Section 6, Bug 2).
- Offers to schedule a site visit, capturing name and phone number - skipping the phone number question entirely if it is already known from the call itself (the dialed number on outbound, caller ID on inbound).
- Recognizes when a caller is busy or in a hurry and stops probing for details immediately, saving whatever partial lead info already exists rather than losing it or annoying the caller.

- Answers FAQs: office hours, documents needed, brokerage fees, current offers.
- Ends gracefully on any real signal - 'thank you', 'that's all', or 'I'm in a hurry' - with a warm closing line and the shop's callback number.
- Every call is recorded in full (dual-channel) as a durable artifact.
- A small web dashboard shows captured leads with live stats, not just raw JSON.

## 4. Architecture

Caller <-> Twilio (telephony) <-> FastAPI backend, which in turn calls:

- Speech-to-text: faster-whisper, self-hosted, GPU-accelerated when available, falls back to CPU automatically.
- Conversation: Claude Haiku 4.5, with a system prompt that embeds the small listings/FAQ dataset directly (faster than per-turn tool-call lookups on a live phone call), plus save_lead and end_call tools for structured actions.
- Text-to-speech: edge-tts - free, Indian-accented voices, no billing setup required (unlike Google/Azure Cloud TTS, which need a billing-enabled account even for their free tier).
- Storage: SQLite for leads, written to disk the moment a lead is captured, not held only in memory.

Why this stack: every component is free or self-hosted, proving the product works before committing to recurring infrastructure cost. This is a deliberate tradeoff, documented honestly in Section 8 (Known Limitations), not an oversight.

## 5. Build Process, Step By Step

- Scaffolding - FastAPI backend, modular folders per pipeline stage (STT, TTS, LLM, telephony, storage) so each piece could be built and tested independently.
- Sample data - placeholder property listings and FAQ data, since no real client inventory existed. Kept language-neutral so the LLM translates/phrases it on the fly instead of duplicating content per language.
- Speech-to-text - faster-whisper, verified against real audio before trusting it. Tested smaller/faster Whisper model variants for speed, but they broke Hindi accuracy badly (one variant transcribed Hindi speech into Urdu script instead of Devanagari) - kept the larger, accurate model.
- Text-to-speech - edge-tts, verified with a round-trip test: synthesized Hindi speech fed back into the STT module came back with the correct meaning intact.
- Conversation logic - system prompt plus save_lead/end_call tools. Found and fixed a real bug during testing: the agent waited for a fully confirmed visit time before saving a lead, meaning a caller who hung up early would be lost entirely. Fixed to save as soon as the essentials are known.
- Pipeline wiring - one shared function chains STT to LLM to TTS, reused by both local

testing and the later telephony integration.

- Lead capture - SQLite, keyed by call ID, written to disk immediately.

- Latency tuning - benchmarked each pipeline stage separately. Found speech-to-text dominant and root-caused it: Whisper's encoder processes a fixed 30-second window per call regardless of actual clip length. Optimized via GPU acceleration and greedy decoding instead of beam search.

- Outbound calling - the same conversational agent can place calls via the Twilio REST API pointed at the same webhook, with a direction-aware system prompt so the framing matches (see Section 3).

- The webhook-timeout problem - the single hardest architecture decision, detailed fully in Section 6.

- Deployment - pushed to a public GitHub repo, deployed to Railway. Found and fixed a port mismatch and stripped 500+ MB of dead-weight GPU dependencies (see Section 7).

- Dashboard - a small web UI at the deployed root URL showing captured leads with basic stats.

- Debugging from real usage - after deploying, ran real live calls in both directions and found bugs that only show up under real conditions, not in scripted tests (Section 6).

## 6. The Hardest Problem: Twilio's Synchronous Webhook

Twilio's voice webhooks are synchronous: reply too slowly and Twilio gives up and plays a generic error to the caller. This stack's full pipeline - speech-to-text, an LLM call, then text-to-speech - realistically takes 8 to 14 seconds on a free-tier stack. That does not fit inside a single webhook response.

**The solution:**

The recording-handler endpoint responds to Twilio instantly (under 100ms) and kicks off the actual STT->LLM->TTS processing as a background task. A separate polling endpoint holds the caller with short silent pauses interleaved with brief, varied, bilingual 'thinking' filler phrases (not one robotic repeated line) until the background result is ready, then plays the real answer. This is the same pattern real production voice systems use for slow backends behind synchronous webhooks.

## 7. Real Bugs Found And Fixed Via Live Calls

These bugs did not show up in text-based testing - they only appeared once real phone calls were placed and the recordings analyzed. This is the strongest evidence of engineering rigor in the whole project: not just building something that runs, but verifying it actually works and fixing what doesn't.

**Bug 1: Token truncation silently corrupted conversation state.**

A tight max_tokens limit could cut off a tool call mid-generation. Anthropic's API sets

stop_reason to 'max_tokens' in that case, not 'tool_use' - code that only checked for 'tool_use' treated the truncated response as a normal finished turn, leaving a broken tool-call block in conversation history that crashed every subsequent turn in that call with a 400 error. Fixed by raising the token budget and adding a defensive check that drops any truncated tool_use block before it can corrupt history.

**Bug 2: The agent claimed a listing was 'within budget' when it was nearly double the caller's stated budget.**

The model was doing loose semantic matching without actually comparing the numbers - it said a property priced at 1 crore was 'right around' a 90-lakh budget. Fixed by requiring an explicit arithmetic comparison before any budget-fit claim, with instructions to state a genuine mismatch honestly rather than smooth it over with vague reassuring language.

**Bug 3: A latency optimization silently dropped an entire caller turn.**

The recording's silence-detection timeout was trimmed from 3 seconds to 2 for a snappier feel. In a real call, the caller paused briefly mid-sentence and Twilio cut the recording before they finished speaking - speech-to-text received empty audio, and the call ended with a false 'I didn't catch that, goodbye.' Reverted to 3 seconds: a dropped turn is a far worse failure than one extra second of wait.

**Bug 4: The 'thinking' filler played in the wrong language.**

It was hardcoded to Hindi regardless of what language the caller was actually speaking, which read as a language mismatch to English-speaking callers. Fixed by making fillers bilingual and selecting per-call based on the caller's own detected language.

**Bug 5: A leftover network hiccup could kill an entire call.**

A transient DNS/connection error while downloading the caller's recording from Twilio's API raised an unhandled exception that ended the whole turn with 'something went wrong.' Fixed by adding retries with short backoff before giving up - a brief network blip should never be able to end a live call.

## 8. Deployment

Code pushed to a public GitHub repository, deployed to Railway (chosen for its simple CLI-driven deploy flow and generous free tier).

**Bug found during deployment: port mismatch.**

Railway assigns its own dynamic port via a PORT environment variable. The public domain was initially configured to route to a hardcoded port that did not match what the app actually bound to, producing a generic 502 error with no obvious cause. Traced by checking the actual runtime logs for the port the app printed on startup, then updating the domain's target port to match.

**Bug found during deployment: wasted build size.**

The production requirements file initially included GPU-acceleration packages (500+ MB) that

are pure dead weight on Railway, which has no GPU - the speech-to-text module already falls back to CPU automatically. Removed from production requirements and documented as an opt-in local addition instead.

## 9. Known Limitations (Stated Honestly)

**Latency.**

End-to-end turn time is realistically 8-14 seconds on this free/local stack. Whisper's encoder processes a fixed-size window regardless of clip length, which is the dominant, largely untunable cost on this architecture. A paid streaming STT/TTS stack would get this well under 2 seconds - a real cost tradeoff, not an oversight.

**Placeholder business data.**

The listings and FAQ data are illustrative, not a real property inventory - this project was built speculatively, without an existing client relationship.

**Outbound calling compliance.**

Outbound calling is built and functionally works, but real cold-list marketing calls in India require TRAI telemarketer registration and DND-list scrubbing, which is out of scope for this project.

**In-memory call state.**

Active call sessions live in a Python dict, not a persistent store - fine for a single-process demo, but would need Redis or similar to survive a restart or scale past one worker process.

## 10. Skills This Project Demonstrates

- Real-time multimodal pipeline engineering - chaining speech-to-text, an LLM, and text-to-speech under real latency constraints.
- Working around a hard platform constraint with an async background-task and poll pattern, not just hoping the pipeline would be fast enough.
- Multilingual conversational AI, including correct grammatical gender agreement for a female TTS voice in Hindi.
- Honest latency engineering - measured the real number, understood exactly why, and designed around the constraint instead of hiding it.
- Debugging from production signal, not just unit tests - every bug in Section 7 was found by analyzing real recorded calls.
- Cost-conscious architecture, including catching real inefficiencies (500MB of dead-weight dependencies) before they became habit.
- Independent product scoping with no client spec.
- Full deployment ownership: GitHub, Railway, DNS/port configuration, environment secrets, telephony webhook configuration.

- Structured data extraction from unstructured voice, tied to a business outcome (a usable lead record).